

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Полтавська політехніка
імені Юрія Кондратюка»

Кафедра комп'ютерних та інформаційних технологій і систем

Звіт
з лабораторної роботи №3
з дисципліни «Технології на платформі NET»,
тема: «Робота із базами даних в .NET. Використання бібліотеки Entity
Framework.»

Виконав:

Студент групи 404-ТН

Плаксюк Я.М.

Перевірила:

Викладач каф. КІТС Тютюнник П.Б.

Полтава, 2025

Завдання

1. Створити проєкт {Назва тематики}.Infrastructure. У новоствореному проєкті створити папку Models. У створеній папці додати модель(набір класів), яка відповідає створеній у першій лабораторній роботі моделі, та містить всередині себе властивості. Наприклад, якщо існував у першій лабораторній клас Bus, у папці Models необхідно створити клас BusModel. Отримана модель має обов'язково мати зв'язки один-до-одного та один-до-багатьох. За додавання зв'язків багато-до-багатьох можна отримати додаткові бали. Для наслідування більш пріоритетно застосовувати підхід Таблиця на тип (Table-per-Type).
2. Створити клас {Назва тематики}Context, який наслідуватиметься від класу DbContext та опише схему бази даних для обраної тематики застосовуючи Entity Framework та анотації або Fluent API на вибір, Fluent API більш пріоритетно.
3. Створити міграцію використовуючи Entity Framework. Створену міграцію застосувати до реляційної бази даних. Можна використовувати будь-яку реляційну СУБД, як MS SQL, PostgreSQL, MySQL, тощо. Також слід використати SQLite, коли немає можливості розробляти та здавати Лабораторну роботу на одній системі. При використанні SQLite файл бази даних необхідно додати до гіт репозиторія.
4. Реалізувати шаблон проєктування Репозиторій, *{Назва тематики}Context* який буде використовувати клас *{Назва тематики}Context* для досьупу до даних із БД:

```
public interface IRepository<T> where T : class
{
    Task<T> GetByIdAsync(int id);
    Task<IEnumerable<T>> GetAllAsync();
    Task AddAsync(T entity);
    Task Update(T entity);
    Task Delete(T entity);
}
```

```
}
```

5. Оновити асинхронну версію дженерік CRUD сервісу, щоб він використовував репозиторій для доступу до даних:

```
public interface ICrudServiceAsync<T>
```

```
{
```

```
    public Task<bool> CreateAsync(T element);
```

```
    public Task<T> ReadAsync(Guid id);
```

```
    public Task<IEnumerable<T>> ReadAllAsync();
```

```
    public Task<IEnumerable<T>> ReadAllAsync(int page, int amount);
```

```
    public Task<bool> UpdateAsync(T element);
```

```
    public Task<bool> RemoveAsync(T element);
```

```
    public Task<bool> SaveAsync();
```

```
}
```

6. Модифікуйте консольний застосунок, щоб він використовував оновлену версію CRUD сервісу та дані із бази даних.

Виконання

Завдання 1

```
Zoo > Zoo.Infrastructure > Models > AnimalModel.cs
1  namespace Zoo.Infrastructure.Models
2  {
3      public class AnimalModel
4      {
5          public Guid Id { get; set; }
6          public string Name { get; set; }
7          public int Age { get; set; }
8          public double Weight { get; set; }
9
10         public FoodModel Food { get; set; }
11
12         public Guid ZooKeeperModelId { get; set; }
13         public ZooKeeperModel ZooKeeper { get; set; }
14     }
15 }
16
```

Рис. 1 – Доданий файл AnimalModel.cs

```

Zoo > Zoo.Infrastructure > Models > BirdModel.cs
1 namespace Zoo.Infrastructure.Models
2 {
3     public class BirdModel : AnimalModel
4     {
5         public double WingSpan { get; set; }
6         public bool CanFly { get; set; }
7         public string BeakType { get; set; }
8     }
9 }

```

Рис. 2 – Доданий файл BirdModel.cs

```

Zoo > Zoo.Infrastructure > Models > FoodModel.cs
1 namespace Zoo.Infrastructure.Models
2 {
3     public class FoodModel
4     {
5         public Guid Id { get; set; }
6         public string Name { get; set; } = string.Empty;
7         public string Type { get; set; } = string.Empty;
8
9         public Guid AnimalModelId { get; set; }
10        public AnimalModel Animal { get; set; } = null!;
11    }
12 }

```

Рис. 3 – Доданий файл FoodModel.cs

```

Zoo > Zoo.Infrastructure > Models > MammalModel.cs
1 namespace Zoo.Infrastructure.Models
2 {
3     public class MammalModel : AnimalModel
4     {
5         public string FurColor { get; set; }
6         public bool IsPredator { get; set; }
7         public string Habitat { get; set; }
8     }
9 }

```

Рис. 4 – Доданий файл MammalModel.cs

```

Zoo > Zoo.Infrastructure > Models > ZooKeeperModel.cs
1 using System.Collections.Generic;
2
3 namespace Zoo.Infrastructure.Models
4 {
5     public class ZooKeeperModel
6     {
7         public Guid Id { get; set; }
8         public string FullName { get; set; }
9         public int ExperienceYears { get; set; }
10        public string Shift { get; set; }
11
12        public ICollection<AnimalModel> Animals { get; set; }
13    }
14 }

```

Рис. 5 – Доданий файл ZooKeeperModel.cs

Завдання 2

```
Zoo > Zoo.Infrastructure > ZooContext.cs
1  using Microsoft.EntityFrameworkCore;
2  using Zoo.Infrastructure.Models;
3
4  namespace Zoo.Infrastructure
5  {
6      public class ZooContext : DbContext
7      {
8          public DbSet<AnimalModel> Animals { get; set; }
9          public DbSet<BirdModel> Birds { get; set; }
10         public DbSet<MammalModel> Mammals { get; set; }
11         public DbSet<ZooKeeperModel> ZooKeepers { get; set; }
12         public DbSet<FoodModel> Foods { get; set; }
13
14         public ZooContext(DbContextOptions<ZooContext> options) : base(options){}
15
16         protected override void OnModelCreating(ModelBuilder modelBuilder)
17         {
18             base.OnModelCreating(modelBuilder);
19
20             modelBuilder.Entity<AnimalModel>().ToTable("Animals");
21             modelBuilder.Entity<BirdModel>().ToTable("Birds");
22             modelBuilder.Entity<MammalModel>().ToTable("Mammals");
23
24             modelBuilder.Entity<AnimalModel>()
25                 .HasOne(a => a.Food)
26                 .WithOne(f => f.Animal)
27                 .HasForeignKey<FoodModel>(f => f.AnimalModelId);
28
29             modelBuilder.Entity<ZooKeeperModel>()
30                 .HasMany(z => z.Animals)
31                 .WithOne(a => a.ZooKeeper)
32                 .HasForeignKey(a => a.ZooKeeperModelId);
33         }
34     }
35 }
```

Рис. 6 – Доданий файл ZooContext.cs

Завдання 3

```
Zoo > Zoo.Console > Program.cs
1  using Microsoft.Extensions.Hosting;
2  using Microsoft.Extensions.DependencyInjection;
3  using Microsoft.Extensions.Configuration;
4  using Zoo.Infrastructure;
5  using Microsoft.EntityFrameworkCore;
6  using System.Threading.Tasks;
7  using System;
8
9  class Program
10 {
11     static async Task Main(string[] args)
12     {
13         var host = CreateHostBuilder(args).Build();
14         await host.RunAsync();
15     }
16
17     public static IHostBuilder CreateHostBuilder(string[] args) =>
18         Host.CreateDefaultBuilder(args)
19             .ConfigureServices((hostContext, services) =>
20             {
21                 var connectionString = hostContext.Configuration.GetConnectionString("DefaultConnection");
22                 services.AddDbContext<ZooContext>(options =>
23                     options.UseNpgsql(connectionString));
24             });
25 }
```

Рис. 7 – Вигляд файлу Program.cs

```

Zoo > Zoo.Console > {} appsettings.json > ...
1  {
2  "ConnectionStrings": {
3    "DefaultConnection": "Host=localhost;Port=5432;Database=zoo_db;Username=postgres;Password=1234"
4  }
5  }
6

```

Рис. 8 – Підключення до бд

Register - Server

General Connection Parameters SSH Tunnel Advanced Post Connection SQL Tags

Name: zoo_db

Server group: Servers

Background: ☐

Foreground: ☐

Connect now?: ☒

Comments:

❗ Either Host name or Service must be specified.

Close Reset Save

Рис. 9 – Створення бд zoo_db

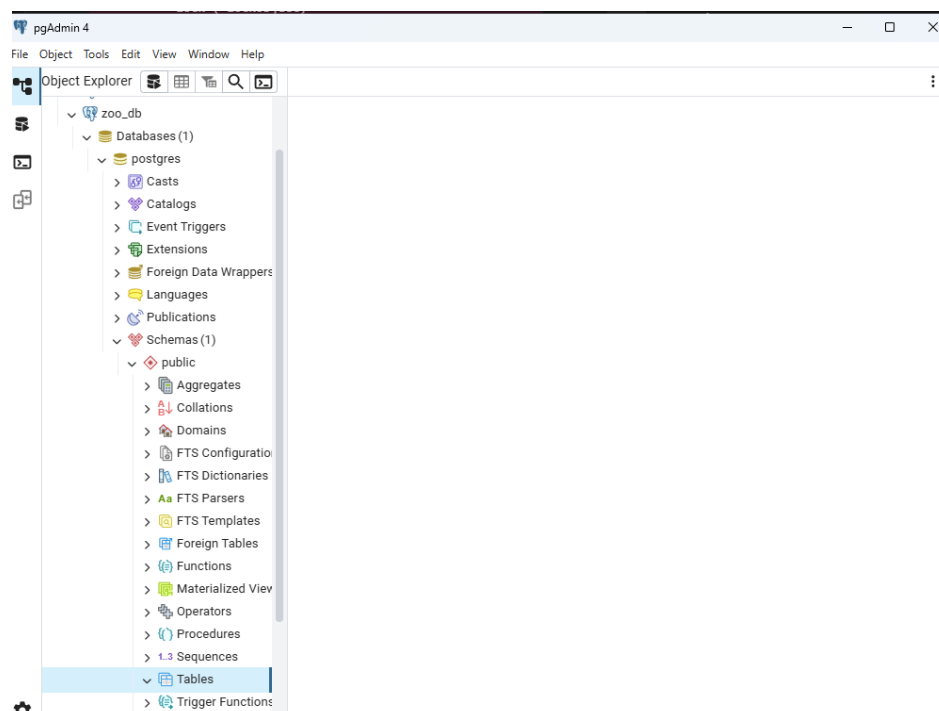


Рис. 10 – Демонстрація того, що вона пуста

```

PS C:\Users\usern\NUPP_NET_2025_404_TN_Plaksiuk_Lab> cd Zoo
PS C:\Users\usern\NUPP_NET_2025_404_TN_Plaksiuk_Lab\Zoo> dotnet dotnet-ef migrations add InitialCreate --project Zoo.Infrastructure --startup-project Zoo.Console
Build started...
Build succeeded.
The Entity Framework tools version '9.0.0' is older than that of the runtime '9.0.1'. Update the tools for the latest features and bug fixes. See https://aka.ms/AAC1fbw for more information.
Done. To undo this action, use 'ef migrations remove'
PS C:\Users\usern\NUPP_NET_2025_404_TN_Plaksiuk_Lab\Zoo> dotnet dotnet-ef database update --project Zoo.Infrastructure --startup-project Zoo.Console
Build started...
Build succeeded.
The Entity Framework tools version '9.0.0' is older than that of the runtime '9.0.1'. Update the tools for the latest features and bug fixes. See https://aka.ms/AAC1fbw for more information.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (12ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT "MigrationId", "ProductVersion"
      FROM "__EFMigrationsHistory"
      ORDER BY "MigrationId";

```

Рис. 11 – Створення папки міграції та оновлення бд

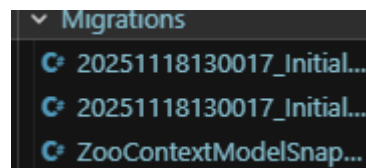


Рис. 12 – Вигляд папки міграції

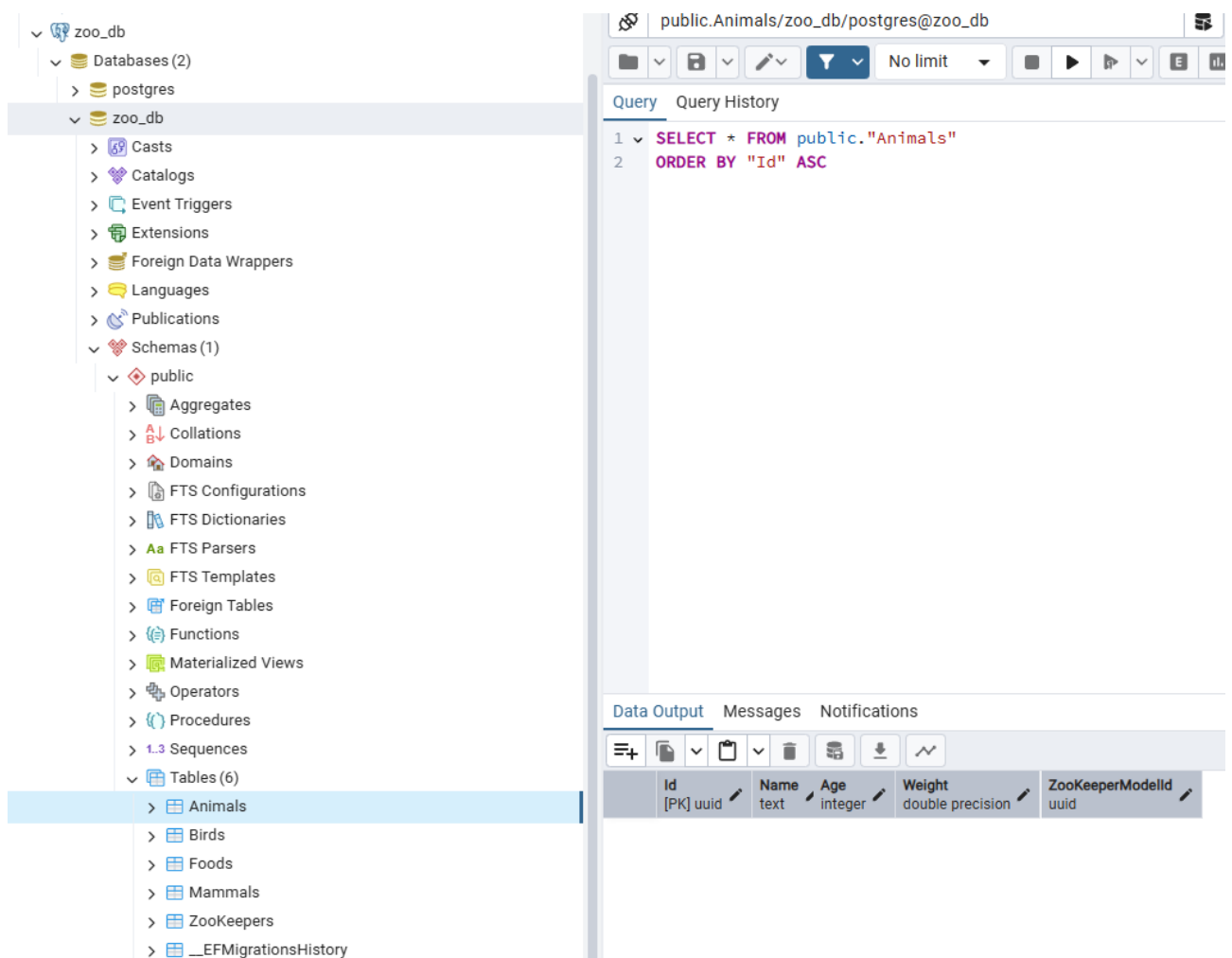


Рис. 13 – Демонстрація того, що таблиці додалися. Таблиця Animals

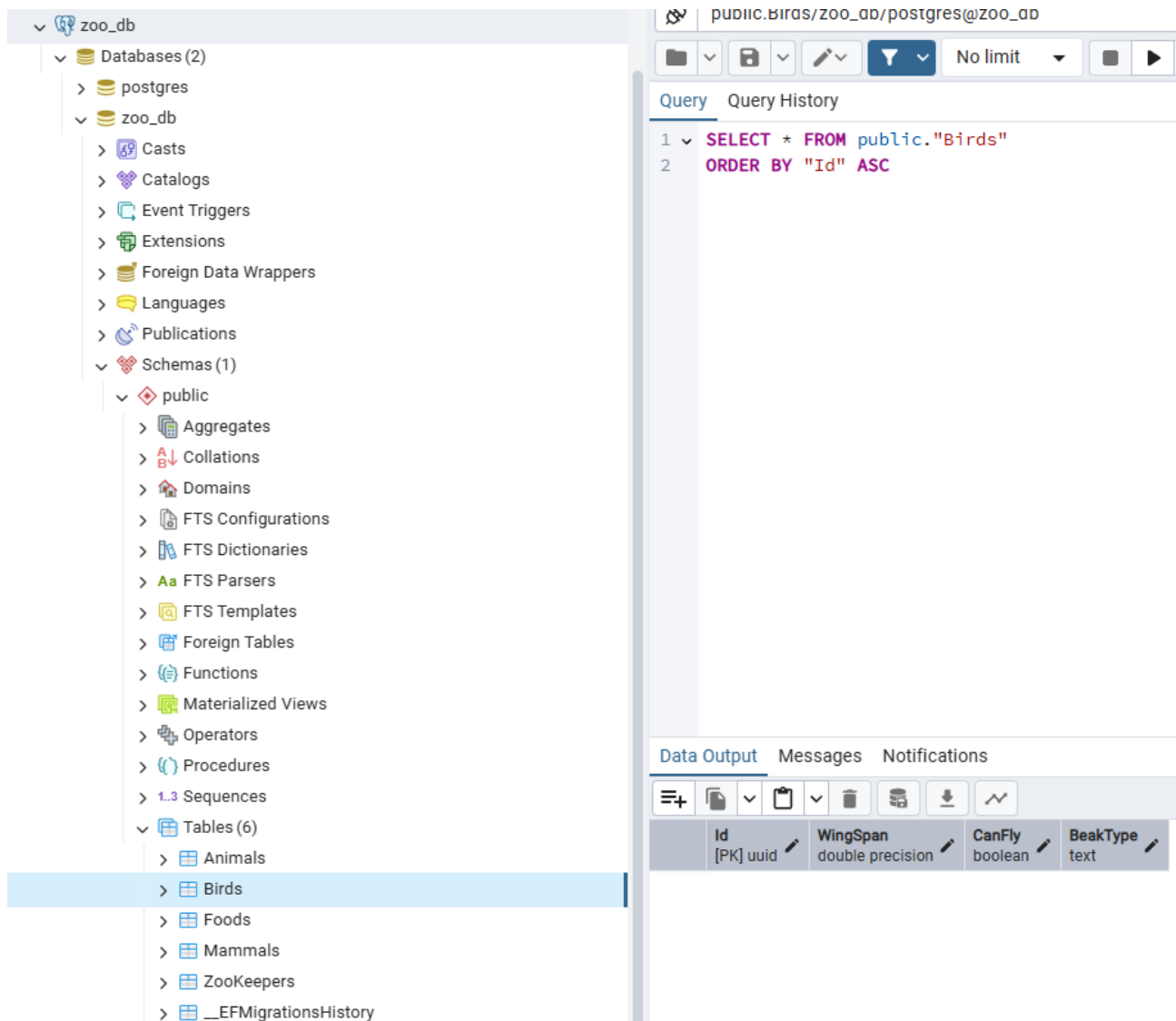


Рис. 14 – Таблиця Birds

Завдання 4

```

Zoo > Zoo.Common > IRepository.cs
1  namespace Zoo.Common
2  {
3      public interface IRepository<T> where T : class
4      {
5          Task<T> GetByIdAsync(Guid id);
6          Task<IEnumerable<T>> GetAllAsync();
7          Task AddAsync(T entity);
8          Task UpdateAsync(T entity);
9          Task DeleteAsync(T entity);
10     }
11 }

```

Рис. 15 – Створення файлу IRepository.cs


```
Zoo > Zoo.Infrastructure > Repository.cs

1  using Microsoft.EntityFrameworkCore;
2  using Zoo.Common;
3
4  namespace Zoo.Infrastructure
5  {
6      public class Repository<T> : IRepository<T> where T : class
7      {
8          private readonly ZooContext _context;
9          private readonly DbSet<T> _dbSet;
10
11         public Repository(ZooContext context)
12         {
13             _context = context;
14             _dbSet = context.Set<T>();
15         }
16
17         public async Task<IEnumerable<T>> GetAllAsync()
18         {
19             return await _dbSet.ToListAsync();
20         }
21
22         public async Task<T> GetByIdAsync(Guid id)
23         {
24             return await _dbSet.FindAsync(id);
25         }
26
27         public async Task AddAsync(T entity)
28         {
29             await _dbSet.AddAsync(entity);
30             await _context.SaveChangesAsync();
31         }
32
33         public async Task UpdateAsync(T entity)
34         {
35             _dbSet.Attach(entity);
36             _context.Entry(entity).State = EntityState.Modified;
37             await _context.SaveChangesAsync();
38         }
39
40         public async Task DeleteAsync(T entity)
41         {
42             if (_context.Entry(entity).State == EntityState.Detached)
43             {
44                 _dbSet.Attach(entity);

```

Рис. 16 – Створення файлу Repository.cs

Завдання 5

```
Zoo > Zoo.Common > ICrudServiceAsync.cs
1  using System;
2  using System.Collections.Generic;
3  using System.Threading.Tasks;
4
5  namespace Zoo.Common
6  {
7      public interface ICrudServiceAsync<T>
8      {
9          Task<bool> CreateAsync(T element);
10         Task<T> ReadAsync(Guid id);
11         Task<IEnumerable<T>> ReadAllAsync();
12         Task<IEnumerable<T>> ReadAllAsync(int page, int amount);
13         Task<bool> UpdateAsync(T element);
14         Task<bool> RemoveAsync(T element);
15     }
16 }
```

Рис. 17 – Перероблений ICrudServiceAsync.cs

```
Zoo.Common > CrudServiceAsync.cs
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace Zoo.Common
{
    public class CrudServiceAsync<T> : ICrudServiceAsync<T> where T : class
    {
        private readonly IRepository<T> _repository;

        public CrudServiceAsync(IRepository<T> repository)
        {
            _repository = repository;
        }

        public async Task<bool> CreateAsync(T element)
        {
            try
            {
                await _repository.AddAsync(element);
                return true;
            }
            catch
            {
                return false;
            }
        }

        public async Task<T> ReadAsync(System.Guid id)
        {
            return await _repository.GetByIdAsync(id);
        }

        public async Task<IEnumerable<T>> ReadAllAsync()
        {
            return await _repository.GetAllAsync();
        }

        public async Task<IEnumerable<T>> ReadAllAsync(int page, int amount)
        {
            var allItems = await _repository.GetAllAsync();
            return allItems.Skip((page - 1) * amount).Take(amount);
        }

        public async Task<bool> UpdateAsync(T element)
        {
            try
            {
                await _repository.UpdateAsync(element);
                return true;
            }
            catch
            {
                return false;
            }
        }

        public async Task<bool> RemoveAsync(T element)
        {
            try
            {
                await _repository.RemoveAsync(element);
                return true;
            }
            catch
            {
                return false;
            }
        }
    }
}
```

Рис. 18 – Перероблений CrudServiceAsync.cs

Завдання 6

```
oo.Console > Program.cs

using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Configuration;
using Microsoft.EntityFrameworkCore;
using System;
using System.Threading.Tasks;
using Zoo.Common;
using Zoo.Infrastructure;
using Zoo.Infrastructure.Models;
using System.Linq;

var host = Host.CreateDefaultBuilder(args)
    .ConfigureServices((hostContext, services) =>
    {
        string connectionString = hostContext.Configuration.GetConnectionString("DefaultConnection");
        services.AddDbContext<ZooContext>(options => options.UseNpgsql(connectionString));

        services.AddScoped(typeof(IRepository<>), typeof(Repository<>));
        services.AddScoped(typeof(ICrudServiceAsync<>), typeof(CrudServiceAsync<>));
    })
    .Build();

await RunDemo(host.Services);

Console.ReadLine();

static async Task RunDemo(IServiceProvider services)
{
    using (var scope = services.CreateScope())
    {
        var zooKeeperService = scope.ServiceProvider.GetRequiredService<ICrudServiceAsync<ZooKeeperModel>>();
        var animalService = scope.ServiceProvider.GetRequiredService<ICrudServiceAsync<AnimalModel>>();

        var newKeeper = new ZooKeeperModel
        {
            Id = Guid.NewGuid(),
            FullName = "Ярослав Жук",
            ExperienceYears = 8,
            Shift = "Денна"
        };

        await zooKeeperService.CreateAsync(newKeeper);
        Console.WriteLine($"Доглядач {newKeeper.FullName} доданий до БД.");

        Console.WriteLine("\nСтворення нової тварини ");
        var newAnimal = new MammalModel
        {

```

Рис. 19 – Перероблений Program.cs

```
PS C:\Users\user\WMPF.NET_2025_404_TN\Plaskiuk_Lab\Zoo> dotnet run --project Zoo.Console
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (31ms) [Parameters=[@p0='?' (DbType = Guid), @p1='?' (DbType = Int32), @p2='?', @p3='?'], CommandType='Text', CommandTimeout='30']
      INSERT INTO "ZooKeepers" ("Id", "ExperienceYears", "FullName", "Shift")
      VALUES (@p0, @p1, @p2, @p3);
Доглядач Ярослав Жук доданий до БД.

Створення нової тварини
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (6ms) [Parameters=[@p0='?' (DbType = Guid), @p1='?' (DbType = Int32), @p2='?', @p3='?' (DbType = Double), @p4='?' (DbType = Guid), @p5='?' (DbType = Guid), @p6='?' (DbType = Boolean)], CommandType='Text', CommandTimeout='30']
      INSERT INTO "Animals" ("Id", "Age", "Name", "Weight", "ZooKeeperModelId")
      VALUES (@p0, @p1, @p2, @p3, @p4);
      INSERT INTO "Mammals" ("Id", "FurColor", "Habitat", "IsPredator")
      VALUES (@p5, @p6, @p7, @p8);
Тварина Вепрідь Бурий додана до БД
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT z."Id", z."ExperienceYears", z."FullName", z."Shift"
      FROM "ZooKeepers" AS z
Знайдено глядачів у БД 4
ID: e1a6cb2e-7e06-4a70-a29f-753b47bf7dba Ім'я: Ярослав Жук
ID: 78292314-dc92-47bf-87fe-78885a1bb9e0 Ім'я: Ярослав Жук
ID: 88a0c906-bc5f-4042-964e-7f73fc3190fe Ім'я: Ярослав Жук
ID: ec987eb9-2fa5-4a98-8970-3103def52442 Ім'я: Ярослав Жук
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (2ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT a."Id", a."Age", a."Name", a."Weight", a."ZooKeeperModelId", b."BeakType", b."CanFly", b."WingSpan", m."FurColor", m."Habitat", m."IsPredator", CASE
      WHEN m."Id" IS NOT NULL THEN "MammalModel"
      WHEN b."Id" IS NOT NULL THEN "BirdModel"
      END AS "Discriminator"
      FROM "Animals" AS a
      LEFT JOIN "Birds" AS b ON a."Id" = b."Id"
      LEFT JOIN "Mammals" AS m ON a."Id" = m."Id"
Знайдено тварин у БД 4
ID: 73e2cde7-0894-4893-b5fb-f781b220e95b Ім'я: Вепрідь Бурий Доглядач: 78292314-dc92-47bf-87fe-78885a1bb9e0
ID: 5272e090-5ee4-4b08-b0d1-8953f57b093d Ім'я: Вепрідь Бурий Доглядач: 88a0c906-bc5f-4042-964e-7f73fc3190fe
ID: 8c9b0180-32c8-45a9-b8ce-28596af1b06a Ім'я: Вепрідь Бурий Доглядач: e1a6cb2e-7e06-4a70-a29f-753b47bf7dba
ID: e6f44c59-d6e1-427d-9703-fd43c5aaa3db Ім'я: Вепрідь Бурий Доглядач: ec987eb9-2fa5-4a98-8970-3103def52442
```

Рис. 20 – Запуск проекту

Query Query History

```
1 SELECT * FROM public."Animals"
2 ORDER BY "Id" ASC
```

Data Output Messages Notifications

	Id [PK] uuid	Name text	Age integer	Weight double precision	ZooKeeperModelId uuid
1	52726090-5eca-4b88-80d3-8953f57b69...	Ведмідь Бурий	4	250	88a0c906-0c5f-4042-964c-7f73fc319bfe
2	73e2cde7-0894-4893-b5fb-f781b220e0...	Ведмідь Бурий	4	250	78292314-dc92-47bf-87fe-70885a1bb9...
3	8c9b0109-32c8-45a9-80ce-29596af10b...	Ведмідь Бурий	4	250	e1a6cb2e-7e06-4a70-a29f-753b47bf7dba
4	e6f44c59-d6e1-427d-9703-fd43c5aaa3db	Ведмідь Бурий	4	250	ec987eb9-2fa5-4a98-8970-3103def52442

Рис. 21 – Демонстрація того, що інформація додалася успішно

Query Query History

```
1 SELECT * FROM public."ZooKeepers"
2 ORDER BY "Id" ASC
```

Data Output Messages Notifications

	Id [PK] uuid	FullName text	ExperienceYears integer	Shift text
1	78292314-dc92-47bf-87fe-70885a1bb9...	Ярослав Жук	8	Денна
2	88a0c906-0c5f-4042-964c-7f73fc319bfe	Ярослав Жук	8	Денна
3	e1a6cb2e-7e06-4a70-a29f-753b47bf7dba	Ярослав Жук	8	Денна
4	ec987eb9-2fa5-4a98-8970-3103def52442	Ярослав Жук	8	Денна

Рис. 22 – Демонстрація того, що інформація додалася успішно

The screenshot displays a database management interface. On the left, a tree view shows the database structure for 'zoo_db'. The 'public' schema is expanded, showing various database objects. The 'Tables (6)' section is highlighted, and the 'Mammals' table is selected. On the right, a query window shows the following SQL query:

```
1 SELECT * FROM public."Mammals"  
2 ORDER BY "Id" ASC
```

Below the query window, the 'Data Output' tab is active, displaying the results of the query in a table format:

	Id [PK] uuid	FurColor text	IsPredator boolean	Habitat text
1	52726090-5eca-4b88-80d3-8953f57b69...	Бурий	true	Ліс
2	73e2cde7-0894-4893-b5fb-f781b220e0...	Бурий	true	Ліс
3	8c9b0109-32c8-45a9-80ce-29596af10b...	Бурий	true	Ліс
4	e6f44c59-d6e1-427d-9703-fd43c5aaa3db	Бурий	true	Ліс

Рис. 23 – Демонстрація того, що інформація додалася успішно

Висновок

У ході лабораторної роботи я створив папку Models де створив файли з моделями. Створив клас, котрий наслідується та опису схему бд. Потім створив свою Postgres бд, відповідно до моєї теми. Та у ході подальших маніпуляцій, зробив міграцію та заніс дані у бд.